

File: InLineTimerExamples/CountingFrames.py

```
FRAMES_PER_SECOND = 30 # takes 1/30th of a second for each frame
TIMER_LENGTH = 2.5
--- snip ---
timerRunning = False
```

This code shows what happens when the user clicks the Start button:

```
if startButton.handleEvent(event):
    timerRunning = True
    nFramesElapsed = 0 # initialize a counter
    nFramesToWait = int(FRAMES_PER_SECOND * TIMER_LENGTH)
    startButton.disable()
    timerMessage.show()
```

The program calculates that it should wait for 75 frames (2.5 seconds $\times$ 30 frames per second), and we set `timerRunning` to `True` to indicate that the timer has started. Inside the main loop, we use this code to check for when the timer ends:

```
if timerRunning:
    nFramesElapsed = nFramesElapsed + 1 # increment the counter
    if nFramesElapsed >= nFramesToWait:
        startButton.enable()
        timerMessage.hide()
        print('Timer ended by counting frames')
        timerRunning = False
```

When the timer ends, we re-enable the Start button, hide the message, and reset the `timerRunning` variable. (If you prefer, you could set the count to the number of frames to wait and count down to zero instead.) This approach works fine, but it is tied to the program's frame rate.

Timer Event

As a second approach, we'll take advantage of pygame's built-in timer. Pygame allows you to add a new event to the event queue—this is known as *posting* an event. Specifically, we'll ask pygame to create and post a timer event. We only need to specify how far into the future we want the